

Assignment Management System

Database Design Plan | Python + Tkinter + SQLite

This document describes the database design for the M.Tech Assignment Management Application. The system supports two assignment modes — individual submissions by a single student, and group submissions by a team of students — within the same application, using a single, unified schema rather than duplicating tables for each mode.

The design targets SQLite (via Python's built-in `sqlite3` module), which is the natural fit for a Tkinter desktop application: no server setup, a single portable `.db` file, and full SQL support.

1. Design Overview

The schema has eight tables. They fall into four logical groups:

- **Identity & enrollment** — `USERS`, `COURSES`, `ENROLLMENTS`. Who exists in the system, what courses run, and which students are enrolled in which course.
- **Assignment definition** — `ASSIGNMENTS`. What work has been set, by whom, for which course, and whether it is individual or group-based.
- **Group formation** — `GROUPS`, `GROUP_MEMBERS`. How students are organized into teams for a specific group assignment.
- **Submission & evaluation** — `SUBMISSIONS`, `GRADES`. What was handed in, by an individual or a group, and how it was scored.

1.1 The core problem this design solves

The central design challenge is: *an assignment can be submitted by one student, or by a group of students — how do we model that without building two separate submission systems?* The solution used here is a single `SUBMISSIONS` table with two nullable foreign keys, `student_id` and `group_id`. Exactly one of the two is filled in per row, depending on the assignment's type:

- If `ASSIGNMENTS.type = 'INDIVIDUAL'`, every submission for it has `student_id` set and `group_id` left `NULL`.
- If `ASSIGNMENTS.type = 'GROUP'`, every submission has `group_id` set and `student_id` left `NULL`.

This keeps grading, file storage, and reporting logic in one place instead of two parallel code paths for individual and group work.

2. Table-by-Table Design

2.1 USERS

Stores every person who logs into the system — students, faculty, and (optionally) an admin. A single table with a role column avoids duplicating login/auth logic per role.

Field	Type	Constraint	Description
user_id	INTEGER	PK, AUTOINCREMENT	Unique identifier for the user
name	TEXT	NOT NULL	Full name
email	TEXT	UNIQUE, NOT NULL	Used as login identifier
password_hash	TEXT	NOT NULL	Hashed password (never store plain text)
role	TEXT	NOT NULL	'student', 'faculty', or 'admin'
created_at	TIMESTAMP	DEFAULT now	Account creation time

2.2 COURSES

Represents a course/subject offered in a semester, owned by one faculty member.

Field	Type	Constraint	Description
course_id	INTEGER	PK, AUTOINCREMENT	Unique identifier for the course
course_code	TEXT	UNIQUE, NOT NULL	e.g. 'CS501'
course_name	TEXT	NOT NULL	e.g. 'Distributed Systems'
faculty_id	INTEGER	FK → USERS. user_id	Course owner/instructor

2.3 ENROLLMENTS

A join table linking students to the courses they are taking. Needed because a student takes many courses, and a course has many students (many-to-many).

Field	Type	Constraint	Description
enrollment_id	INTEGER	PK, AUTOINCREMENT	Unique identifier
student_id	INTEGER	FK → USERS. user_id	The enrolled student
course_id	INTEGER	FK → COURSES. course_id	The course enrolled in
enrolled_at	TIMESTAMP	DEFAULT now	When enrollment happened

2.4 ASSIGNMENTS

The task itself. The 'type' field is the switch that determines whether submissions for this assignment will reference a student or a group.

Field	Type	Constraint	Description
assignment_id	INTEGER	PK, AUTOINCREMENT	Unique identifier
course_id	INTEGER	FK → COURSES.course_id	Course this belongs to
title	TEXT	NOT NULL	Assignment title
description	TEXT	—	Instructions / brief
type	TEXT	NOT NULL	'INDIVIDUAL' or 'GROUP'
max_marks	REAL	NOT NULL	Maximum achievable score
due_date	DATETIME	NOT NULL	Submission deadline
created_by	INTEGER	FK → USERS.user_id	Faculty who created it
created_at	TIMESTAMP	DEFAULT now	Creation timestamp

2.5 GROUPS

A team formed for one specific group assignment. Groups are scoped per-assignment (not persistent across the whole course) — matching how group work is typically assigned in college courses, where teams are re-formed for each project.

Field	Type	Constraint	Description
group_id	INTEGER	PK, AUTOINCREMENT	Unique identifier
assignment_id	INTEGER	FK → ASSIGNMENTS.assignment_id	Which assignment this group is for
group_name	TEXT	NOT NULL	e.g. 'Group 3' or a student-chosen name
created_at	TIMESTAMP	DEFAULT now	When the group was formed

2.6 GROUP_MEMBERS

Join table linking students to the group(s) they belong to. Needed because a group has multiple students, and (in principle) a student could be in different groups for different assignments.

Field	Type	Constraint	Description
group_id	INTEGER	PK (composite), FK → GROUPS.group_id	The group

Field	Type	Constraint	Description
student_id	INTEGER	PK (composite), FK → USERS. user_id	A member of that group

2.7 SUBMISSIONS

One row per submitted piece of work. Exactly one of student_id / group_id is populated, matching the parent assignment's type (see Section 1.1).

Field	Type	Constraint	Description
submission_id	INTEGER	PK, AUTOINCREMENT	Unique identifier
assignment_id	INTEGER	FK → ASSIGNMENTS.assignment_id	Which assignment this answers
student_id	INTEGER	FK → USERS. user_id, NULLABLE	Set only for individual assignments
group_id	INTEGER	FK → GROUPS.group_id, NULLABLE	Set only for group assignments
file_path	TEXT	NOT NULL	Path to the uploaded file on disk
submitted_at	TIMESTAMP	DEFAULT now	Submission timestamp
status	TEXT	—	'submitted' / 'late' (can be derived, not stored)

2.8 GRADES

Evaluation result for a submission, kept as its own table (rather than columns on SUBMISSIONS) so that grading history and the grader's identity are cleanly separated from the act of submitting.

Field	Type	Constraint	Description
grade_id	INTEGER	PK, AUTOINCREMENT	Unique identifier
submission_id	INTEGER	FK → SUBMISSIONS.submission_id	Submission being graded
marks_obtained	REAL	NOT NULL	Score given
feedback	TEXT	—	Optional comments from faculty
graded_by	INTEGER	FK → USERS. user_id	Faculty who graded it
graded_at	TIMESTAMP	DEFAULT now	Grading timestamp

3. How the Tables Relate

- **USERS** → **COURSES** (one-to-many): one faculty member teaches many courses.
- **USERS** ↔ **COURSES** via **ENROLLMENTS** (many-to-many): a student enrolls in many courses; a course has many students.
- **COURSES** → **ASSIGNMENTS** (one-to-many): one course has many assignments.
- **ASSIGNMENTS** → **GROUPS** (one-to-many, group assignments only): one group assignment can have several groups, one per team.
- **GROUPS** ↔ **USERS** via **GROUP_MEMBERS** (many-to-many): a group has several student members; a student can belong to different groups over time.
- **ASSIGNMENTS** → **SUBMISSIONS** (one-to-many): one assignment receives many submissions (one per student, or one per group).
- **USERS / GROUPS** → **SUBMISSIONS**: a submission belongs to exactly one student OR one group, never both (enforced by a CHECK constraint — see Section 5).
- **SUBMISSIONS** → **GRADES** (one-to-one): each submission has at most one grade record.

4. Key Design Decisions & Why

4.1 Why one **USERS** table instead of separate Student/Faculty tables?

A single table with a `role` column keeps authentication logic identical for every login, and avoids repeating email/password columns in three places. If faculty later need extra fields (department, designation), add a separate `FACULTY_PROFILE` table keyed on `user_id` rather than duplicating the whole user record.

4.2 Why is **GROUPS** scoped to a single assignment?

Modeling a group as belonging to one assignment (rather than to the course as a whole) matches how group work is usually assigned — teams are formed per project. If your course instead uses fixed, persistent teams across the whole semester, **GROUPS** should be linked to the course instead of the assignment, with a separate join table connecting groups to the assignments they are used for.

4.3 Why is **GRADES** a separate table, not columns on **SUBMISSIONS**?

Keeping grading separate cleanly records *who* graded the work and *when*, independent of the submission event itself, and leaves room for re-grading history later without altering the submission record.

4.4 Why not store files inside the database?

SQLite can store binary blobs, but for a desktop app it is simpler and faster to save uploaded files to disk (e.g. under `uploads/<assignment_id>/`) and store only the file path as text in `SUBMISSIONS.file_path`. This keeps the database small and lets you open/preview files directly from the filesystem.

4.5 Why compute 'status' instead of storing it?

A submission's status ('on time' vs 'late') can always be derived by comparing `submitted_at` to the assignment's `due_date` at read time. Storing it as a separate column risks it going stale if the due date is ever edited after submissions come in.

5. SQLite Schema (CREATE TABLE statements)

This can be run once at application startup to initialize a fresh database file.

```
CREATE TABLE USERS (  
    user_id      INTEGER PRIMARY KEY AUTOINCREMENT,  
    name         TEXT NOT NULL,  
    email        TEXT NOT NULL UNIQUE,  
    password_hash TEXT NOT NULL,  
    role         TEXT NOT NULL CHECK (role IN ('student','faculty','admin')),  
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE COURSES (  
    course_id    INTEGER PRIMARY KEY AUTOINCREMENT,  
    course_code  TEXT NOT NULL UNIQUE,  
    course_name  TEXT NOT NULL,  
    faculty_id   INTEGER NOT NULL REFERENCES USERS(user_id)  
);  
  
CREATE TABLE ENROLLMENTS (  
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    student_id    INTEGER NOT NULL REFERENCES USERS(user_id),  
    course_id     INTEGER NOT NULL REFERENCES COURSES(course_id),  
    enrolled_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE (student_id, course_id)  
);  
  
CREATE TABLE ASSIGNMENTS (  
    assignment_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    course_id     INTEGER NOT NULL REFERENCES COURSES(course_id),  
    title         TEXT NOT NULL,  
    description   TEXT,  
    type         TEXT NOT NULL CHECK (type IN ('INDIVIDUAL','GROUP')),  
    max_marks     REAL NOT NULL,  
    due_date      DATETIME NOT NULL,  
    created_by    INTEGER NOT NULL REFERENCES USERS(user_id),  
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE GROUPS (  
    group_id      INTEGER PRIMARY KEY AUTOINCREMENT,  
    assignment_id INTEGER NOT NULL REFERENCES ASSIGNMENTS(assignment_id),  
    group_name    TEXT NOT NULL,  
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE GROUP_MEMBERS (  
    group_id      INTEGER NOT NULL REFERENCES GROUPS(group_id),  
    student_id    INTEGER NOT NULL REFERENCES USERS(user_id),  
    PRIMARY KEY (group_id, student_id)  
);  
  
CREATE TABLE SUBMISSIONS (  
    submission_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    assignment_id  INTEGER NOT NULL REFERENCES ASSIGNMENTS(assignment_id),  
    student_id     INTEGER REFERENCES USERS(user_id),  
    group_id       INTEGER REFERENCES GROUPS(group_id),  
    file_path      TEXT NOT NULL,  
    submitted_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    CHECK (  
        (student_id IS NOT NULL AND group_id IS NULL) OR
```

```

        (student_id IS NULL AND group_id IS NOT NULL)
    )
);

CREATE TABLE GRADES (
    grade_id        INTEGER PRIMARY KEY AUTOINCREMENT,
    submission_id    INTEGER NOT NULL UNIQUE REFERENCES SUBMISSIONS(submission_id),
    marks_obtained   REAL NOT NULL,
    feedback         TEXT,
    graded_by        INTEGER NOT NULL REFERENCES USERS(user_id),
    graded_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

6. Notes for the Tkinter Implementation

- Use Python's built-in `sqlite3` module — no external DB server needed, single portable `.db` file.
- Store uploaded assignment files on disk, e.g. `uploads/<assignment_id>/<student_or_group_id>_<filename>`, and keep only the path string in `SUBMISSIONS.file_path`.
- Hash passwords (e.g. with `hashlib` + a salt, or `bcrypt`) — never store plain-text passwords.
- Enable foreign key enforcement explicitly in SQLite: `conn.execute('PRAGMA foreign_keys = ON')` on every connection, since SQLite disables it by default.
- Wrap multi-step operations (e.g. creating a group and adding members) in a single transaction so a crash mid-way doesn't leave a group with zero members.
- Keep a single `database.py` module with connection handling and CRUD functions, and call it from your Tkinter screens — keeps UI code free of raw SQL.